Week 00: Get Set Up

Project: Predicting Heart Failure Survival · Notebook: notebooks/00_setup_check.ipynb

This week you set up your computer for the project. You won't touch any data yet. If you've never used a terminal or installed Python before, this week is for you, and it's meant to be done slowly.

By the end of this week you will have:

- Python 3.14 installed
- A project folder with the right structure
- A private "virtual environment" holding the packages you need
- Jupyter Notebook running, with a notebook that proves everything works

Plan for about 1 to 2 hours, most of it waiting for downloads and installs. You need a computer you can install software on, an internet connection, and roughly 1 GB of free space.

About This Project

Over six weeks you will build a model that predicts whether a heart-failure patient died during follow-up, and test a research claim that two blood measurements are enough to predict it. Each week has one notebook, goals, steps, written questions, a checklist, and a **Check Your Answers** section. Small differences from the expected numbers are normal.

Week	Topic	Notebook
00	Get set up	`00_setup_check.ipynb`
01	Explore the data	`01_eda.ipynb`
02	Survival curves and the leakage problem	`02_survival.ipynb`
03	Prepare the data and build your first model	`03_baseline.ipynb`
04	Compare many models and find key features	`04_compare_models.ipynb`
05	Test the paper's claim and tune	`05_paper_and_tuning.ipynb`
06	Final test, interpretation and report	`06_final.ipynb`

Rules for the Whole Project

- **Never edit files in `data/raw/`.** Save anything you change in `data/processed/`, so you can always start over.
- **Never use the `time` column to predict death.** You will learn why in Week 02.
- **Hide the test set until Week 06.** Make every decision using the training data only.

- **Explain as you go.** Before each step, add a Markdown cell saying what you are doing; after each result, say what you found.
- **Restart and Run All before you hand anything in.** This is a learning project, not a medical tool.

The Big Picture

Here is what you're about to do, so the steps make sense:

1. Install **Python**, the language your code is written in.
2. Open a **terminal**, a window where you type commands.
3. Make a **project folder** with places for your data, notebooks and results.
4. Create a **virtual environment**, a private copy of Python for this project, so installing things here can't break anything else.
5. Install the **packages** (libraries) the project uses.
6. Start **Jupyter Notebook**, where you'll write and run your code.
7. Run a **check notebook** to make sure it all works.

Step 1: Install Python 3.14

Everyone uses the same version, so we're all on the same page when something goes wrong. Go to python.org/downloads and download the latest 3.14 release (3.14.x). If the main download button offers a newer version than 3.14, look further down the page, or on Python's release pages, for the latest 3.14.x and use that, since that's the version this guide was checked with.

Windows

1. Run the installer.
2. On the first screen, tick **"Add Python to PATH"** before you click anything else. The wording may differ slightly in newer installers. This is the most common mistake. If you miss it, nothing in this guide will work.
3. Click **Install Now**.

Mac

Run the installer you downloaded and accept the defaults.

Linux (Ubuntu or Debian)

Many Linux versions don't offer Python 3.14 through apt. Install the system Python first, then check which version you got:

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
python3 --version
```

If it shows Python 3.12 or newer, that works for this project. If it's older, download Python 3.14 from python.org/downloads and follow the instructions there for Linux.

Check it worked (after you open a terminal in Step 2):

- Windows: `python --version`
- Mac / Linux: `python3 --version`

You should see Python 3.14.something. If you see a different version, see Troubleshooting at the end.

Step 2: Open a Terminal

A terminal lets you control your computer by typing commands instead of clicking. It looks plain, but you only need a handful of commands.

- **Windows:** press the Windows key, type `cmd`, and press Enter. This opens Command Prompt.
- **Mac:** press Cmd + Space, type `Terminal`, and press Enter.
- **Linux:** press Ctrl + Alt + T.

You'll see a line ending in `>` (Windows) or `$` or `%` (Mac/Linux). That's the **prompt**. It's waiting for a command. You type after it and press Enter.

Step 3: Learn the Few Commands You Need

What you want to do	Windows (Command Prompt)	Mac / Linux
Show where you are	`cd`	`pwd`
List what's in this folder	`dir`	`ls`
List hidden files too	`dir /a`	`ls -a`
Go into a folder	`cd foldername`	`cd foldername`
Go up one folder	`cd ..`	`cd ..`
Go to your home folder	`cd %USERPROFILE%`	`cd ~`
Make a folder	`mkdir foldername`	`mkdir foldername`
Show a text file	`type file.txt`	`cat file.txt`
Clear the screen	`cls`	`clear`

Shortcuts worth knowing:

- Press **Tab** to auto-complete a folder or file name
- Press the **up arrow** to bring back your last command
- Press **Ctrl + C** to stop a command that's running

> **Careful:** deleting files in the terminal has no undo and no recycle bin. You won't need to delete anything this week.

Try it. Run these one at a time and read what each one prints.

Windows:

```
cd %USERPROFILE%
mkdir dev
cd dev
cd
dir
```

Mac / Linux:

```
cd ~
mkdir dev
cd dev
pwd
ls
```

You now have a folder called `dev` where your projects will live. If `mkdir dev` says the folder already exists, that's fine.

Step 4: Build Your Project Folder

You'll make this structure yourself, so you know where everything is:

```
heart-failure-project/
├── requirements.txt
├── data/
│   ├── raw/          (the original download, never edited)
│   └── processed/    (data you clean or change)
├── notebooks/        (your Jupyter notebooks)
├── reports/
│   └── figures/      (saved plots)
└── results/          (saved models and result tables)
```

Spell the folder names exactly like this, in lowercase, with no spaces. Your notebooks will look for files at these exact paths, so a typo like `Data` or `raw_data` will break things later.

From inside `dev`, run:

Windows:

```
mkdir heart-failure-project
cd heart-failure-project
mkdir data\raw data\processed notebooks reports\figures results
```

Mac / Linux:

```
mkdir heart-failure-project
cd heart-failure-project
mkdir -p data/raw data/processed notebooks reports/figures results
```

Create requirements.txt

This file lists the packages the project needs, one per line. Make it with a text editor:

- **Windows:** run `notepad requirements.txt`. When Notepad asks whether to create a new file, click Yes.
- **Mac / Linux:** run `nano requirements.txt`. (Quit with Ctrl + X, press Y to save, and press Enter to exit.)

Paste in exactly this, save, and close:

```
numpy
pandas
scipy
scikit-learn
xgboost
lightgbm
matplotlib
seaborn
notebook
```

What each one is for:

- `numpy`, `pandas`: working with numbers and tables
- `scipy`: statistical tests
- `scikit-learn`: the models and scoring tools (it also brings `joblib`, which you'll use to save your model)
- `xgboost`, `lightgbm`: two extra, powerful tree-based models
- `matplotlib`, `seaborn`: making charts
- `notebook`: Jupyter Notebook itself

In Week 02 you'll add one more package, `lifelines`, for survival curves.

Check your structure

Still inside `heart-failure-project`:

- Windows: `tree /f`
- Mac / Linux: `ls -R`

You should see the folders above and `requirements.txt`. If the file shows up as `requirements.txt.txt`, rename it: `ren requirements.txt.txt requirements.txt` (Windows) or `mv requirements.txt.txt requirements.txt` (Mac / Linux).

Step 5: Create a Virtual Environment

A virtual environment is a private copy of Python for this one project. Anything you install goes into it and nowhere else.

Make sure you're inside `heart-failure-project` (use the "where am I" command from Step 3), then:

Windows:

```
python -m venv .venv
.venv\Scripts\activate.bat
```

Mac / Linux:

```
python3 -m venv .venv
source .venv/bin/activate
```

Creating it takes a few seconds. After activating, your prompt should start with `(.venv)`. That's how you know it's on.

- The folder `.venv` starts with a dot, which hides it in some file views. That's normal. Use `dir /a` or `ls -a` to see it.
- You have to activate it every time you open a new terminal. Closing the terminal turns it off. To turn it off on purpose, type `deactivate`.

Step 6: Install the Packages

With `(.venv)` showing in your prompt:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

This can take a few minutes. When it finishes, check:

```
pip list
```

You should see numpy, pandas, scipy, scikit-learn, xgboost, lightgbm, matplotlib, seaborn and notebook in the list, along with many others they depend on.

> **If you don't see `(.venv)` in your prompt, stop.** Activate the environment first, or the packages will install in the wrong place.

Step 7: Start Jupyter Notebook

From `heart-failure-project`, with `(.venv)` showing:

```
jupyter notebook
```

Your web browser opens with a page showing your project's files. If it doesn't open, look in the terminal for a link beginning with `http://localhost:8888/` or `http://127.0.0.1:8888/` and paste it into your browser.

Leave the terminal window open while you work. It's running Jupyter, and closing it stops everything.

Make your first notebook:

1. In the browser, click the `notebooks` folder.
2. Click **New** (top right), then **Python 3 (ipykernel)**.
3. Click the title ("Untitled") at the top and rename it to `00_setup_check`.

To stop Jupyter when you're done: save your notebook, close the browser tab, then click on the terminal and press **Ctrl + C** (it may ask you to confirm).

Step 8: Run the Check Notebook

In the first cell of `00_setup_check`, type this and press **Shift + Enter** to run it. (You can also click the Run button, the ▶ in the toolbar at the top. Ctrl + Enter runs a cell and stays on it.)

```
import sys
print(sys.executable)
```

The path it prints should contain `.venv`. That proves the notebook is using your project's environment.

In the next cell:

```
import numpy as np
import pandas as pd
import scipy
import sklearn
import xgboost
import lightgbm
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns

print("Python", sys.version.split()[0])
print("numpy", np.__version__)
print("pandas", pd.__version__)
print("scipy", scipy.__version__)
print("scikit-learn", sklearn.__version__)
print("xgboost", xgboost.__version__)
print("lightgbm", lightgbm.__version__)
print("matplotlib", matplotlib.__version__)
print("seaborn", sns.__version__)
```

In a third cell, make a table and a plot:

```
df = pd.DataFrame({"x": [1, 2, 3, 4, 5], "y": [2, 4, 5, 4, 6]})
display(df)

df.plot(x="x", y="y", kind="line", marker="o")
plt.title("My first plot")
plt.show()
```

Save with **Ctrl + S**. If you see the path with `.venv`, the version numbers, and a plot, you're set up.

Every Time You Come Back

Open a terminal and run these three steps.

Windows:

```
cd %USERPROFILE%\dev\heart-failure-project
.venv\Scripts\activate.bat
jupyter notebook
```

Mac / Linux:

```
cd ~/dev/heart-failure-project
source .venv/bin/activate
jupyter notebook
```

Troubleshooting

Problem	What to do
`'python' is not recognized` (Windows)	Python isn't on your PATH. Run the installer again and make sure **Add Python to PATH** is ticked on the first screen. (If the installer offers Modify, choose it, then Next, and look for an option about the PATH or environment variables.) Close the terminal and open a new one. If Windows opens the Microsoft Store when you type `python`, search Windows for "Manage app execution aliases" and turn off the Python entries.
`python --version` shows the wrong version	You may have more than one Python. On Windows, try `py -3.14 --version`. If that works, use `py -3.14 -m venv .venv` in Step 5. On Mac / Linux, try `python3.14 --version`.
`(.venv)` doesn't appear after activating	Check that you're inside `heart-failure-project`. On Windows, check you used backslashes. On Mac / Linux, check you used `source`.
`pip` or `jupyter` is not recognized	The environment isn't active. Look for `(.venv)`, and activate it.
`ModuleNotFoundError: No module named 'pandas'` in a notebook	Jupyter was started without the environment. Close it (Ctrl + C in the terminal), activate `.venv`, and start `jupyter notebook` again. Check that the `sys.executable` path includes `.venv`.
Jupyter doesn't open in the browser	Copy the `http://localhost:8888/...` (or `http://127.0.0.1:8888/...`) link from the terminal into your browser.
`jupyter notebook` says the port is in use	Another Jupyter is running. Close it, or use the new link in the terminal, since it will use a different number.
The file is called `requirements.txt.txt`	Windows hid the real extension. Rename it as shown in Step 4. You can also turn on View → File name extensions in File Explorer.
I installed a package but the notebook can't find it	Restart the kernel: in the notebook menu, **Kernel → Restart Kernel**.
A package fails to install	Read the last lines of the error. Make sure `(.venv)` is active and pip is upgraded (Step 6), then run `pip install -r requirements.txt` again. If one package still fails, ask your instructor and note which one.

Deliverable

- [ ] `tree /f` (Windows) or `ls -R` (Mac / Linux) shows the folder structure from Step 4
- [ ] `(.venv)` shows in your prompt when you activate it
- [ ] `pip list` shows the packages
- [ ] `00_setup_check.ipynb` runs from top to bottom: the path contains `.venv`, the versions print, and the plot appears

Next Week

In Week 01, you'll download the heart failure dataset and start exploring it.